

David Nguyen

CSEN 296B-2 — AI-Driven Software Development

Lab 1.2: Initial Requirement Set

After choosing the Supabase architecture from Submission 1, I used Copilot Chat to generate an initial requirement baseline. I used the Role + Context + Constraint + Goal pattern from class to frame the prompt:

```
"Act as a senior software analyst. I am defining requirements for AstraNotes, a secure and modular note-taking system built with Next.js 14 and Supabase. This is a solo quarter project with a 10-week timeline. Generate an initial requirement baseline with functional requirements, non-functional requirements, and security or governance requirements. Write them clearly enough that each one could later become a backlog item, a design decision, a prototype target, a test case, or an acceptance criterion."
```

The first pass gave me 15+ requirements but a lot of them were vague (“the app should be fast”, “data should be secure”). I went back and asked it to add measurable thresholds, specify what “done” looks like for each one, and make sure they covered failure handling and privacy boundaries—not just happy-path features. Here’s the refined set of 14.

Functional Requirements (8)

FR-01 — Rich-text note editing with Markdown

Users can create, edit, and delete notes using a rich-text editor that supports Markdown (headings H1–H3, bold, italic, inline code, code blocks, ordered/unordered lists, links). The editor renders a live preview as the user types. All changes persist to Supabase Postgres on save.

Test: User types `## Hello \*\*world\*\*` and sees it rendered as a styled heading with bold. User deletes a note and sees a confirmation dialog first. Content round-trips to the database without losing formatting.

FR-02 — Folder and tag organization

Users can create, rename, and delete folders. Every note belongs to exactly one folder (default: “Uncategorized”). Users can assign zero or more tags per note. The sidebar displays the folder tree and a tag filter panel.

Test: Moving a note between folders updates sidebar counts immediately. Clicking a tag filters notes across all folders. Deleting a folder with notes prompts the user to move or delete them.

FR-03 — Full-text search

Users can search across all their notes by keyword. Search uses Postgres tsvector/tsquery indexing on the title and body fields. Results are ranked by relevance and highlight matching terms in a snippet preview.

Test: Searching “project kickoff” returns matching notes with highlighted fragments. Results return within 500ms for up to 1,000 notes. An empty search shows all notes.

FR-04 — Email/password authentication with secure sessions

Users register and log in via email/password through Supabase Auth. Sessions use secure HTTP-only cookies via the Supabase Next.js Auth helper. Login errors do not reveal whether the email exists in the system.

Test: New user signs up, confirms via email link, and can log in. Wrong password returns a generic “Invalid login credentials” message. Session refreshes silently after 30 minutes of inactivity; full re-auth required after 7 days.

FR-05 — Share notes with permission levels

A note owner can share a note with another authenticated user by email. The owner selects read-only or read-write permission. Shared notes appear in the recipient’s “Shared with me” view. The owner can revoke access at any time.

Test: Read-only recipient can view but the editor is disabled. Read-write recipient can edit and save. Revoking access removes the note from the recipient’s shared view on next load.

FR-06 — Real-time collaborative editing

When two users have the same note open in read-write mode, edits from one appear on the other’s screen in near real-time via Supabase Realtime channels.

Test: User A types a sentence; User B sees it within 2 seconds without refreshing. Both users can type simultaneously without overwriting each other. If the connection drops, the editor stays functional locally and shows a “Sync paused” banner.

FR-07 — Note version history and restore

Every explicit save creates a version snapshot in a separate versions table. Users can open a history panel, view past versions with timestamps, preview any version read-only, and restore it. Restoring creates a new version entry so the action is itself reversible.

Test: User saves 3 times, opens history, sees 3 entries newest-first. Previewing shows content at that point in time. Restoring replaces current content and adds a 4th version entry.

FR-08 — Export as Markdown

Users can export a single note as a .md file download preserving all Markdown source. A bulk export option downloads all notes in a folder as a .zip of .md files.

Test: Exported .md renders correctly in any Markdown viewer. Bulk export of a 5-note folder produces a .zip with 5 correctly named .md files.

Non-Functional Requirements (3)

NFR-01 — Page load performance

The authenticated dashboard reaches First Contentful Paint in under 1.5 seconds on broadband. The note list is rendered via Next.js server components so the initial HTML includes real content, not a loading spinner.

Test: Lighthouse desktop performance score ≥ 90 . Cold load with 50 notes shows the list in under 1.5s measured by FCP.

NFR-02 — Mobile responsiveness

All primary views are usable at 375px viewport width. The sidebar collapses into a hamburger menu. The editor toolbar wraps and all touch targets are at least 44×44px.

Test: At 375px the sidebar is behind a menu icon. No horizontal scrollbar on any primary view. All toolbar buttons meet 44px minimum.

NFR-03 — Testability by design

The storage layer, access control logic, and sync service are separated into distinct modules so each can be unit-tested independently. Integration tests can run against a local Supabase instance without depending on production data.

Test: Running the test suite with a seeded local Supabase instance produces passing results for note CRUD, RLS enforcement, and version history without touching production. Each module has at least one isolated unit test.

Security, Privacy & Governance Requirements (3)

SPR-01 — Row-level security as the trust boundary

Supabase RLS policies are enabled on notes, folders, tags, note_shares, and note_versions. A user can only access rows where they are the owner or an explicitly granted collaborator. No API route or server action bypasses RLS. The owner_id column is the single trust boundary for all data access.

Test: Integration test where User B queries User A's note ID and gets an empty result or 403. RLS policies are version-controlled in a Supabase migration file. A CI script that checks for RLS-disabled tables fails the build if any are found.

SPR-02 — Encryption in transit and at rest

All client-server communication uses HTTPS/TLS. The Vercel deployment enforces HTTPS with automatic cert management. Supabase encrypts Postgres data at rest via AES-256. No endpoint is reachable over plain HTTP.

Test: Hitting the root domain over HTTP returns a 301 to HTTPS. Supabase dashboard confirms disk encryption is active. SSL Labs scan returns an A rating or above.

SPR-03 — Audit log for shared note access

Every view, edit, or permission change on a shared note writes an event to an audit_log table with actor user ID, action type, target note ID, and timestamp. The audit_log table is append-only—RLS blocks UPDATE and DELETE operations on it.

Test: User B opens a shared note; an audit_log row appears with action="view." Owner revokes access; a row appears with action="revoke." Attempting to delete an audit_log row via the API returns a permission error.

Total: 14 requirements—8 functional, 3 non-functional, 3 security/privacy/governance. The biggest improvement from the refinement passes was getting the AI to think about failure handling and privacy boundaries in the requirements themselves, not just in the architecture. The first draft treated security as an afterthought ("data should be encrypted"). After I pushed it to specify where the trust boundary lives, what's append-only, and what happens when a connection drops, the requirements actually became something I could turn into sprint work

and write tests against.